YZM304 Derin Öğrenme — 4. Proje Taslağı

📌 Proje Başlığı

Oyun Ekran Görüntülerinden Oyun Tespiti: Transfer Learning Tabanlı CNN ve Vision Transformer Mimarilerinin Karşılaştırmalı Analizi

(İngilizce: "Video Game Identification from Gameplay Screenshots: A Comparative Analysis of Transfer Learning Based CNN and Vision Transformer Architectures")

🎯 Proje Amacı

Bu çalışmanın amacı, popüler video oyunlarına ait ekran görüntülerini otomatik olarak sınıflandırmak için farklı derin öğrenme mimarilerini karşılaştırmaktır. Çalışma; klasik CNN (ResNet50), verimli modern CNN (EfficientNetB0) ve Transformer tabanlı (Vision Transformer) mimarilerin gameplay ekran görüntüsü sınıflandırmasındaki performansını doğruluk, model boyutu ve eğitim süresi açısından analiz etmektedir.

Gerçek dünya uygulamaları:

- Streaming platformlarında (Twitch, YouTube) otomatik oyun kategorilendirme
- Oyun içerik moderasyonu
- Spor yayıncılığı ve e-spor analitiği
- Oyun keşif/öneri sistemleri

📋 Proje Gereksinimleri (Ders Dokümanına Göre)

☑ **Platform yayını:** Proje GitHub üzerinde yayımlanacak (Kaggle/HuggingFace opsiyonel)
☑ **Blog yazısı:** Projede kullanılan yöntemi veya projenin tamamını anlatan blog yazısı
☑ **Sunum:** 13 veya 14. haftada PowerPoint sunumu
☑ **IMRAD formatı:** Sunum IMRAD formatında (Introduction, Methods, Results, And Discussion)
☑ **Değerlendirme ağırlığı:** Proje değerlendirmesinin %33'ü, genel not ortalamasının %10'u

---

🗂 Veri Seti

Birincil Kaynak

Gameplay Images Dataset (Kaggle)

- **Link:** https://www.kaggle.com/datasets/aditmagotra/gameplay-images

- **Alternatif (HuggingFace):** https://huggingface.co/datasets/Bingsu/Gameplay_Images

Veri Seti Özellikleri

Özellik	Değer
Toplam görsel sayısı	10.000
Sınıf sayısı	10
Sınıf başına görsel	1.000 (dengeli)
Görüntü boyutu	640 × 360 (PNG)
Toplam boyut	~2.5 GB
Veri kaynağı	YouTube gameplay videolarından frame extraction

Sınıflar (10 Popüler Oyun)

1. Among Us
2. Apex Legends
3. Fortnite
4. Forza Horizon
5. Free Fire
6. Genshin Impact
7. God of War
8. Minecraft
9. Roblox
10. Terraria

✅ Veri Setinin Proje İçin Uygunluk Kontrol Listesi

VS Code agent'ın kontrol etmesi gereken kriterler:

- [ ] **Klasör yapısı:** Her sınıf için ayrı klasör var mı? (ImageFolder formatı)
- [ ] **Görsel sayısı:** Her sınıfta 1.000 görsel var mı? (dengeli mi?)
- [ ] **Görsel bütünlüğü:** Bozuk/açılamayan PNG var mı?
- [ ] **Görsel boyutları:** Tümü 640×360 mı, yoksa farklı boyutlar var mı?
- [ ] **Duplicate kontrolü:** Aynı kareden birden fazla var mı? (perceptual hash ile)
- [ ] **Renk kanalları:** RGB mi, RGBA mı? (preprocessing gerekebilir)
- [ ] **Görsel kalitesi:** Frame extraction artifact'leri (blur, watermark, subtitle) var mı?

☐ **Sınıf ayrılabilirliği:** Görsel olarak benzer sınıflar var mı? (Fortnite vs Apex — ikisi de battle royale)

Uygunluk Kararı İçin Kriterler

PROJE UYGUN: Aşağıdaki şartlar sağlanırsa:

- Toplam en az 8.000 temiz görsel kaldıysa
- Sınıf başına minimum 700 görsel varsa
- En az 8 sınıf kullanılabilir durumdaysa

VERİ SETİ DEĞİŞTİRİLMELİ: Eğer:

- Görsel sayısı beklenenin çok altındaysa
- Görsellerin %20'sinden fazlası bozuksa
- Sınıflar aşırı dengesizse

🧠 Kullanılacak Modeller

1. ResNet50 (Klasik CNN Baseline)

- **Yayın yılı:** 2015
- **Parametre sayısı:** ~25 milyon
- **Mimari tipi:** Residual (artık bağlantılı) derin CNN
- **Rolü:** Baseline — diğer modellerin kıyaslanacağı referans
- **Pretrained:** ImageNet ağırlıkları ile transfer learning
- **timm kodu:** `timm.create_model('resnet50', pretrained=True, num_classes=10)`

2. EfficientNetB0 (Modern Verimli CNN)

- **Yayın yılı:** 2019
- **Parametre sayısı:** ~5 milyon
- **Mimari tipi:** Compound scaling ile optimize edilmiş CNN
- **Rolü:** Az parametreyle yüksek performans testi
- **Pretrained:** ImageNet ağırlıkları
- **timm kodu:** `timm.create_model('efficientnet_b0', pretrained=True, num_classes=10)`

3. Vision Transformer (ViT-Base/16)

- **Yayın yılı:** 2020

- **Parametre sayısı:** ~86 milyon
- **Mimari tipi:** Attention tabanlı Transformer
- **Rolü:** CNN vs Transformer paradigma karşılaştırması
- **Pretrained:** ImageNet-21k ağırlıkları
- **timm kodu:** `timm.create_model('vit_base_patch16_224', pretrained=True, num_classes=10)`
- **Alternatif (VRAM yetmezse):** `swin_tiny_patch4_window7_224`

Model Karşılaştırma Matrisi

Özellik	ResNet50	EfficientNetB0	ViT-Base/16
Paradigma	Klasik CNN	Modern CNN	Transformer
Parametre	25M	5M	86M
Yayın yılı	2015	2019	2020
Beklenen eğitim süresi (20 epoch)	~45 dk	~30 dk	~90 dk
Beklenen doğruluk aralığı	%85-92	%87-93	%88-95

🔧 Teknik Yapılandırma

Donanım

- **GPU:** NVIDIA RTX 5070 Ti Laptop (12 GB VRAM)
- **Platform:** Lokal Python ortamı + VS Code

Framework & Kütüphaneler

```
torch>=2.0
torchvision
timm
albumentations
scikit-learn
pandas
numpy
matplotlib
seaborn
Pillow
```

```
tqdm
tensorboard  # opsiyonel
```

Eğitim Hiperparametreleri

Parametre	Değer
Giriş boyutu	224 × 224
Batch size	32 (ViT için 16)
Optimizer	AdamW
Learning rate	1e-4
Weight decay	1e-5
LR Scheduler	CosineAnnealingLR
Loss	CrossEntropyLoss
Epoch sayısı	20 (Early Stopping, patience=5)
Train/Val/Test split	70% / 15% / 15% (Stratified)

Data Augmentation Pipeline

- Resize to 256, RandomCrop 224
- Horizontal Flip (p=0.5)
- Color Jitter (brightness=0.2, contrast=0.2, saturation=0.2)
- Random Rotation (±10°)
- Normalize (ImageNet mean/std: [0.485, 0.456, 0.406] / [0.229, 0.224, 0.225])

📊 Değerlendirme Metrikleri

- **Accuracy** (genel doğruluk)
- **Precision, Recall, F1-Score** (sınıf başına)
- **Macro-F1 ve Weighted-F1**
- **Confusion Matrix** (10x10, görsel olarak)
- **Top-3 Accuracy**
- **Eğitim süresi** (epoch başına ve toplam)
- **Model boyutu** (MB cinsinden)

- **Inference süresi** (görüntü başına ms)
- **FLOPs** (hesaplama karmaşıklığı)

📁 Proje Klasör Yapısı

```
oyun-ekran-siniflandirma/
├── README.md
├── requirements.txt
├── .gitignore
├── data/
│   ├── raw/                      # Ham Gameplay Images dataset
│   ├── processed/                # Filtrelenmiş ve split edilmiş veri
│   │   ├── train/
│   │   ├── val/
│   │   └── test/
│   └── EDA.ipynb                 # Keşifsel Veri Analizi
├── src/
│   ├── __init__.py
│   ├── config.py                 # Hiperparametreler
│   ├── dataset.py                # PyTorch Dataset sınıfı
│   ├── transforms.py             # Augmentation pipeline
│   ├── models.py                 # ResNet50, EfficientNet, ViT
│   ├── train.py                  # Eğitim scripti
│   ├── evaluate.py               # Değerlendirme
│   ├── visualize.py              # Grafikler ve confusion matrix
│   └── utils.py                  # Yardımcı fonksiyonlar
├── notebooks/
│   ├── 01_EDA.ipynb
│   ├── 02_training_resnet50.ipynb
│   ├── 03_training_efficientnet.ipynb
│   ├── 04_training_vit.ipynb
│   └── 05_comparison.ipynb
├── results/
│   ├── models/                   # Eğitilmiş model ağırlıkları (.pth)
│   ├── figures/                  # Confusion matrix, loss/accuracy plots
│   ├── logs/                     # Training logs
│   └── metrics.csv               # Tüm metriklerin özet tablosu
├── presentation/
│   └── IMRAD_slides.pptx
└── blog/
    └── blog_post.md
```

Hafta 1: Veri Hazırlama ve Keşif

- [ ] Gameplay Images dataset'ini indirme
- [ ] VS Code agent ile veri seti uygunluk kontrolü
- [ ] EDA: örnek görseller, sınıf dağılımı, görsel boyut analizi
- [ ] Stratified train/val/test split
- [ ] GitHub repo kurulumu + README

Hafta 2: Baseline Model (ResNet50)

- [ ] Dataset ve DataLoader sınıfı yazımı
- [ ] Augmentation pipeline kurulumu
- [ ] ResNet50 eğitimi ve hiperparametre tuning
- [ ] İlk sonuçların değerlendirilmesi
- [ ] EfficientNetB0 eğitimi

Hafta 3: Vision Transformer ve Karşılaştırma

- [ ] ViT-Base/16 eğitimi
- [ ] 3 modelin karşılaştırmalı metrikleri
- [ ] Confusion matrix ve görselleştirmeler
- [ ] Yanlış sınıflandırılan örneklerin analizi
- [ ] Grad-CAM görselleştirmeleri (opsiyonel)

Hafta 4: Sunum ve Teslim

- [ ] IMRAD formatında PowerPoint sunumu
- [ ] Blog yazısının yazılması
- [ ] GitHub README finalize
- [ ] Sunum provası
- [ ] Sınıfta sunum (13 veya 14. hafta)

🎤 IMRAD Sunum İskeleti

1. **Title Slide** — Proje başlığı, ad, ders
2. **Introduction**
   - Video oyunu endüstrisinin büyüklüğü ve içerik üretimi
   - Otomatik oyun tespiti neden önemli? (streaming, moderasyon, analitik)
   - Problem tanımı ve araştırma sorusu
3. **Methods**

- Gameplay Images dataset tanıtımı (10 sınıf, 10.000 görsel)
- Ön işleme ve augmentation
- 3 model mimarisinin açıklanması
- Eğitim protokolü

4. **Results**
    - Her modelin accuracy/F1 skorları
    - Karşılaştırma tablosu ve grafikler
    - Confusion matrix görselleri
    - Eğitim süresi vs doğruluk analizi
5. **Discussion**
    - Hangi model kazandı ve neden?
    - Karıştırılan sınıflar (Fortnite vs Apex, Fortnite vs Free Fire)
    - Model boyutu vs doğruluk trade-off
    - Sınırlılıklar ve gelecek çalışmalar
6. **Conclusion**
    - Ana bulgular
    - Katkılar
7. **References**

✍️ Blog Yazısı Planı

- **Başlık Önerisi:** "AI Fortnite'ı Apex Legends'tan Ayırt Edebilir mi? Derin Öğrenme ile Oyun Tespiti"
- **Giriş:** Gaming endüstrisi + AI kesişimi, okuyucu kancası
- **Problem:** Ekran görüntüsünden oyun tespiti neden zor? (benzer grafik stilleri)
- **Yöntem:** 3 modelin basit açıklaması (teknik olmayan dille)
- **Sonuçlar:** Grafikler ve confusion matrix
- **Şaşırtıcı gözlemler:** Hangi oyunlar karıştırıldı ve neden?
- **Sonuç ve ileriye dönük çalışmalar**
- **GitHub linki + iletişim**

🤖 VS Code Agent İçin Veri Seti Kontrol Promptu

Agent'a şu prompt'u verebilirsin:

```
Bu projenin amacı 10 popüler video oyunundan alınmış ekran görüntülerini
sınıflandırmak. Kullanılacak dataset: Gameplay Images (10 sınıf × 1000 görsel).

Lütfen aşağıdakileri kontrol et:

1. data/raw/ klasöründe 10 alt klasör var mı?
2. Her alt klasörde yaklaşık 1000 PNG dosyası var mı?
3. Rastgele 20 görseli aç, PIL ile yüklenip yüklenmediğini test et
4. Görsel boyutlarının tutarlılığını kontrol et (640x360 bekleniyor)
5. RGB mi RGBA mı olduğunu kontrol et
6. Her sınıftan 3 örnek görsel göster (matplotlib ile grid)
7. Toplam boyut ve dosya sayısını raporla

Sonuçları markdown tablosu halinde özetle ve projenin
"Oyun Ekran Görüntülerinden Oyun Tespiti" konusu için
uygun olup olmadığına karar ver.
```

🔗 Referans Linkler

Dataset

- Kaggle: https://www.kaggle.com/datasets/aditmagotra/gameplay-images
- HuggingFace: https://huggingface.co/datasets/Bingsu/Gameplay_Images

Literatür (IMRAD raporu için)

- "From Pixels to Titles: Video Game Identification by Screenshots using CNNs" — arXiv:2311.15963
- Roboflow Gaming Datasets: https://universe.roboflow.com/browse/gaming

Teknik Kaynaklar

- timm: https://github.com/huggingface/pytorch-image-models
- PyTorch ImageFolder: https://pytorch.org/vision/stable/datasets.html#imagefolder
- Albumentations: https://albumentations.ai/

📌 Notlar ve Yapılacaklar

- [ ] Dataset uygunluk kontrolü (VS Code agent ile)
- [ ] Konunun eğitmen tarafından onayı
- [ ] GitHub repo oluşturma

- [ ] İlk baseline eğitimi (ResNet50) ve sonuç doğrulama
- [ ] Grad-CAM ekleme kararı (opsiyonel — sunumu güçlendirir)

Son güncelleme: 22 Nisan 2026 Ders: YZM304 Derin Öğrenme — Ankara Üniversitesi